

Unidad Didáctica

Metodologías de Desarrollo, Unidad I

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	5
Modelo de cascada.....	5
Prototípica	7
Evolutivas.....	10
Ágiles.....	12
Bibliografía y Webgrafía	15
Glosario	16



En la disciplina de la ingeniería del software, la elección de una metodología de desarrollo adecuada es crucial para el éxito de cualquier proyecto. Esta unidad, enfocada en las Metodologías de Desarrollo, explorará las diversas formas en que los equipos de desarrollo pueden abordar la creación de sistemas de software, desde enfoques tradicionales hasta métodos modernos y ágiles. Entender estas metodologías no solo proporciona una base sólida para la gestión y ejecución de proyectos, sino que también permite a los futuros ingenieros tomar decisiones informadas que optimicen la eficiencia, la calidad y la adaptabilidad de los productos desarrollados.

El Modelo de Cascada, siendo uno de los enfoques más antiguos y estructurados, servirá como punto de partida. Este modelo lineal, que sigue un flujo secuencial de etapas, ha sido la base de muchos desarrollos exitosos, pero también presenta limitaciones que han dado lugar a la evolución de otras metodologías. A partir de esta base, exploraremos el Modelo Prototípico, que introduce una mayor flexibilidad al permitir la creación de prototipos tempranos que pueden ser iterativamente mejorados en función de la retroalimentación del cliente.

Finalmente, la unidad abordará las metodologías Evolutivas y Ágiles, que han ganado popularidad en los últimos años por su capacidad para adaptarse a entornos cambiantes y requerimientos fluctuantes. Estas metodologías representan un cambio de paradigma en la forma en que se concibe y se gestiona el desarrollo de software,

poniendo énfasis en la entrega continua de valor y en la colaboración cercana con el cliente. A lo largo de esta unidad, los estudiantes estarán expuestos a ejemplos prácticos y casos de estudio que les permitirán comprender cómo seleccionar y aplicar la metodología más adecuada según las necesidades específicas de un proyecto, preparando así el camino para su futuro profesional en la ingeniería del software.

Modelo de cascada

El **Modelo de Cascada** es una de las metodologías de desarrollo de software más antiguas y estructuradas, que sigue un enfoque secuencial en el cual cada fase del desarrollo debe completarse antes de pasar a la siguiente. Este modelo es adecuado para proyectos donde los requisitos son bien comprendidos desde el principio y se espera que permanezcan estables a lo largo del proceso de desarrollo. El modelo es comparado con una cascada de agua, en la que el flujo de una etapa se vierte en la siguiente, sin posibilidad de retorno a la etapa anterior.

Fases del Modelo de Cascada

El Modelo de Cascada se compone de las siguientes fases principales:

- Recolección de Requisitos: En esta fase, se recogen y documentan todos los requisitos del proyecto. Para un contexto nicaragüense, podríamos imaginar una empresa local de comercio electrónico que necesita un sistema de gestión de inventarios. Los requisitos incluirían la capacidad de manejar productos, realizar pedidos, y gestionar las existencias en tiempo real, aspectos que se documentan meticulosamente antes de proceder.
- Diseño del Sistema: A partir de los requisitos, se diseña la arquitectura del sistema. En el ejemplo del sistema de gestión de inventarios, esto incluiría la creación de diagramas de flujo, el diseño de bases de datos, y la planificación de interfaces de usuario que se adecuen a las capacidades tecnológicas y al contexto de uso en Nicaragua, como considerar la conectividad limitada en ciertas áreas rurales.
- Implementación: Aquí se realiza la programación y desarrollo del sistema. Continuando con nuestro ejemplo, los desarrolladores nicaragüenses pueden optar por utilizar tecnologías accesibles y ampliamente conocidas en el país,

como lenguajes de programación populares (Java, Python) y frameworks accesibles que sean sostenibles y mantenibles a nivel local.

- Verificación y Pruebas: Una vez completado el desarrollo, el sistema se somete a pruebas exhaustivas para asegurar que cumple con los requisitos definidos. En este contexto, sería importante realizar pruebas de campo, considerando las variaciones en infraestructura tecnológica entre áreas urbanas y rurales de Nicaragua.
- Mantenimiento: Después de la implementación exitosa del sistema, esta fase se encarga de corregir cualquier problema que pueda surgir y de realizar actualizaciones según sea necesario. Dado que los sistemas en Nicaragua pueden enfrentarse a cambios legislativos o regulatorios, el mantenimiento incluye asegurarse de que el software esté siempre en conformidad con las normativas locales.

Ventajas y Desventajas del Modelo de Cascada

El Modelo de Cascada es ventajoso en proyectos donde los requisitos son bien definidos y poco propensos a cambiar. Sin embargo, su rigidez es una desventaja en entornos donde los requisitos pueden evolucionar durante el desarrollo, lo que puede ser una limitación en proyectos en Nicaragua, donde los entornos empresariales pueden ser volátiles debido a factores económicos o sociales.

Aplicación del Modelo de Cascada en Proyectos Nicaragüenses

Como un ejemplo concreto, se puede analizar el desarrollo de un sistema de gestión escolar para una institución educativa en Nicaragua. Al utilizar el Modelo de Cascada, la escuela podría primero establecer claramente sus necesidades (gestión de estudiantes, calificaciones, asistencia), diseñar un sistema basado en esas necesidades, desarrollarlo, probarlo y, finalmente, implementarlo en sus operaciones diarias. Este enfoque estructurado asegura que el sistema esté bien definido y

documentado desde el principio, lo cual es beneficioso en un contexto donde los recursos de TI son limitados y la planificación debe ser cuidadosa.

El Modelo de Cascada proporciona una estructura clara y bien organizada para el desarrollo de software, especialmente en proyectos con requisitos bien definidos. Sin embargo, es crucial que los estudiantes comprendan tanto sus fortalezas como sus limitaciones, especialmente en el contexto de proyectos nicaragüenses donde la flexibilidad y adaptabilidad pueden ser esenciales. A través del estudio de casos y ejemplos locales, los estudiantes pueden aprender a aplicar este modelo de manera efectiva, entendiendo cuándo es la mejor opción y cuándo podría ser necesario considerar alternativas más ágiles.

Prototípica

El **Modelo Prototípico** es una metodología de desarrollo de software centrada en la creación de prototipos tempranos del sistema que se está desarrollando. Este enfoque permite a los desarrolladores y clientes visualizar y evaluar las funcionalidades del sistema desde las etapas iniciales, facilitando la identificación de requisitos y la realización de ajustes antes de que el producto final sea desarrollado. Esta metodología es especialmente útil en proyectos donde los requisitos no están completamente definidos desde el principio, lo cual es común en muchos contextos, incluidos los proyectos de desarrollo en Nicaragua.

Fases del Modelo Prototípico

El proceso de desarrollo bajo el Modelo Prototípico incluye varias fases clave:

- Identificación de Requisitos Iniciales: Se identifican los requisitos básicos que deben estar presentes en el prototipo. Por ejemplo, si una pequeña empresa nicaragüense de turismo desea un sistema para la gestión de reservas, los requisitos iniciales podrían incluir la capacidad de registrar y gestionar reservas, así como la visualización de disponibilidad en tiempo real.

- Desarrollo del Prototipo Inicial: Se construye un prototipo básico que refleja las características y funcionalidades esenciales del sistema. Este prototipo no es un producto final, sino una representación inicial que puede ser utilizada para explorar y experimentar con diferentes aspectos del sistema. Siguiendo el ejemplo anterior, se podría desarrollar una interfaz sencilla donde los usuarios puedan realizar reservas, visualizar la disponibilidad y proporcionar comentarios.
- Evaluación del Prototipo: El prototipo es presentado al cliente o a los usuarios finales para su evaluación. En el contexto nicaragüense, esto podría implicar pruebas con usuarios en diferentes regiones, considerando las variaciones en el acceso a la tecnología e Internet. La retroalimentación obtenida es crucial para identificar áreas de mejora y ajuste.
- Refinamiento del Prototipo: Basado en la retroalimentación, se realizan iteraciones en el prototipo, refinando y mejorando sus características hasta que el producto cumple con las expectativas del cliente. En cada iteración, se puede agregar más funcionalidad y se mejoran los aspectos visuales y de rendimiento. Este ciclo de desarrollo puede repetirse varias veces hasta que el prototipo evolucione hacia el sistema final.
- Desarrollo e Implementación del Producto Final: Una vez que el prototipo cumple con todos los requisitos, se procede a desarrollar la versión final del sistema, incorporando todas las mejoras y correcciones realizadas durante las iteraciones.

Ventajas y Desventajas del Modelo Prototípico

El Modelo Prototípico ofrece la ventaja de la flexibilidad y la capacidad de adaptación a cambios, lo cual es fundamental en proyectos donde los requisitos pueden no estar claros desde el inicio o donde el entorno puede cambiar rápidamente. En Nicaragua, esto es particularmente relevante en sectores como el turismo o la agricultura, donde las condiciones y necesidades pueden variar considerablemente. Sin

embargo, un desafío del enfoque prototípico es el riesgo de que el cliente se enfoque demasiado en el prototipo, esperando que sea un producto completamente funcional desde las primeras etapas, lo que puede generar malentendidos.

Aplicación del Modelo Prototípico en Proyectos Nicaragüenses

Como ejemplo, consideremos el desarrollo de una aplicación móvil para agricultores en Nicaragua, destinada a proporcionar información en tiempo real sobre precios de productos, condiciones climáticas y técnicas de cultivo. Dado que las necesidades de los agricultores pueden ser diversas y evolucionar con el tiempo, un enfoque prototípico permitiría desarrollar un prototipo inicial que podría ser probado en diferentes comunidades rurales. A través de la retroalimentación directa de los usuarios, se podrían hacer ajustes para asegurar que la aplicación sea verdaderamente útil y accesible, tomando en cuenta factores como la conectividad a Internet en áreas remotas y el nivel de alfabetización digital de los usuarios.

El Modelo Prototípico es una herramienta poderosa para desarrollar sistemas de software en entornos donde la flexibilidad y la adaptabilidad son clave. Al permitir una interacción constante con el cliente y la posibilidad de realizar ajustes durante el proceso de desarrollo, este modelo se adapta bien a la realidad de proyectos en Nicaragua, donde las condiciones y necesidades pueden cambiar rápidamente. Al explorar ejemplos concretos y casos de estudio, los estudiantes aprenderán a aplicar este modelo de manera efectiva, entendiendo cómo y cuándo es más beneficioso en comparación con otras metodologías de desarrollo.

Evolutivas

El **Modelo Evolutivo** es una metodología de desarrollo de software que enfatiza la creación de sistemas a través de la mejora continua y la evolución del producto a lo largo del tiempo. En lugar de seguir un proceso lineal o estrictamente definido desde el principio, el desarrollo evolutivo permite que el software se desarrolle de manera incremental, adaptándose a nuevos requisitos o cambios en el entorno. Este enfoque es especialmente útil en proyectos complejos o de largo plazo, donde los requisitos pueden no estar completamente definidos desde el principio o pueden cambiar durante el desarrollo.

Fases del Modelo Evolutivo

El Modelo Evolutivo se caracteriza por varias fases iterativas:

- Planificación Inicial y Desarrollo Rápido: Se comienza con una planificación inicial que incluye la identificación de los requisitos más importantes y la creación de un plan para desarrollar una primera versión básica del software. Por ejemplo, en un proyecto para desarrollar un sistema de gestión de aguas en Nicaragua, la primera versión podría centrarse en la recolección y monitoreo de datos básicos de uso del agua en comunidades rurales.
- Desarrollo Incremental: En lugar de intentar desarrollar todo el sistema de una vez, el producto se desarrolla en incrementos. Cada incremento representa una versión mejorada del sistema, con nuevas características añadidas y mejoras realizadas en base a la retroalimentación obtenida. En el contexto del sistema de gestión de aguas, esto podría significar agregar nuevas funcionalidades en cada iteración, como la predicción de patrones de uso del agua o la integración de datos climáticos.
- Evaluación y Retroalimentación: Cada versión del software se prueba y se presenta a los usuarios o clientes para su evaluación. Esta fase es crucial en el desarrollo evolutivo, ya que permite identificar rápidamente cualquier problema o ajuste necesario, asegurando que el sistema evolucione de

manera que cumpla con las necesidades reales de los usuarios. En Nicaragua, esto podría implicar pruebas en el campo con usuarios finales, como agricultores o administradores comunitarios.

- Evolución Continua: Basado en la retroalimentación y las pruebas, el sistema sigue evolucionando a través de iteraciones adicionales, mejorando y adaptándose a nuevos requisitos o cambios en el entorno. Esto es particularmente útil en un contexto nicaragüense, donde factores externos como cambios climáticos o normativas gubernamentales pueden influir en los requisitos del sistema.

Ventajas y Desventajas del Modelo Evolutivo

El Modelo Evolutivo es altamente adaptable y permite la incorporación de cambios en el sistema de manera continua, lo que lo hace ideal para proyectos en los que los requisitos no son completamente conocidos desde el principio o pueden cambiar con el tiempo. En Nicaragua, donde los proyectos de software pueden estar sujetos a variaciones en las necesidades de los usuarios o cambios en el entorno tecnológico, esta metodología ofrece la flexibilidad necesaria para enfrentar estos desafíos. Sin embargo, una desventaja es que el desarrollo evolutivo puede prolongar el tiempo de entrega del producto final, ya que implica múltiples iteraciones y ajustes.

Aplicación del Modelo Evolutivo en Proyectos Nicaragüenses

Un ejemplo práctico de aplicación del Modelo Evolutivo en Nicaragua podría ser el desarrollo de un sistema para la gestión de emergencias naturales, como huracanes o terremotos. Dado que las necesidades y los desafíos de la gestión de emergencias pueden cambiar rápidamente, un enfoque evolutivo permitiría desarrollar un sistema inicial básico que luego se iría mejorando en base a experiencias reales y retroalimentación de los usuarios. Por ejemplo, en la primera iteración se podría desarrollar un módulo de notificación de emergencias, seguido por la integración de

mapas interactivos, y finalmente la adición de funciones para coordinar la respuesta de emergencia y el seguimiento de recursos.

El Modelo Evolutivo es una metodología robusta para el desarrollo de software en entornos donde la adaptabilidad y la capacidad de respuesta son críticas. En Nicaragua, donde los proyectos pueden estar sujetos a condiciones cambiantes y a la necesidad de ajustes constantes, este modelo ofrece una forma efectiva de garantizar que el software desarrollado se mantenga relevante y útil a lo largo del tiempo. A través del estudio de este modelo, los estudiantes aprenderán cómo aplicar un enfoque evolutivo en sus propios proyectos, comprendiendo cuándo es más beneficioso y cómo manejar los desafíos que puedan surgir en el proceso.

Ágiles

Las **Metodologías Ágiles** representan un enfoque moderno y flexible para el desarrollo de software, diseñado para adaptarse rápidamente a cambios en los requisitos y a las necesidades del cliente. En lugar de seguir un plan rígido desde el principio hasta el final, las metodologías ágiles se basan en iteraciones cortas, llamadas sprints, donde se desarrollan y entregan incrementos funcionales del software. Este enfoque fomenta la colaboración continua con el cliente y permite ajustar el desarrollo según la retroalimentación obtenida en cada iteración. En el contexto de Nicaragua, donde los entornos empresariales y tecnológicos pueden ser dinámicos, la agilidad es esencial para responder eficazmente a los cambios y a las necesidades del mercado.

Principios Fundamentales de las Metodologías Ágiles

Las Metodologías Ágiles se sustentan en varios principios clave, los cuales guían su aplicación en cualquier proyecto de software:

- Interacciones y Colaboración: En lugar de priorizar herramientas y procesos, las metodologías ágiles enfatizan la importancia de la comunicación directa y la colaboración estrecha entre los equipos de desarrollo y los clientes. En Nicaragua, esto podría implicar una colaboración cercana entre desarrolladores y pequeñas empresas locales, ajustando continuamente el software para satisfacer las necesidades cambiantes del negocio.
- Entregas Frecuentes y Funcionales: Los desarrollos ágiles se organizan en ciclos cortos, donde se entrega un producto funcional al final de cada sprint. Esto permite a los clientes evaluar el progreso y proporcionar retroalimentación inmediata. Por ejemplo, en el desarrollo de una aplicación móvil para el sector agrícola en Nicaragua, un sprint podría enfocarse en la implementación de un módulo para el seguimiento de cultivos, entregando esta funcionalidad rápidamente para su evaluación.
- Adaptabilidad al Cambio: Las metodologías ágiles están diseñadas para ser altamente adaptables, permitiendo cambios en los requisitos incluso en etapas avanzadas del desarrollo. En el contexto nicaragüense, donde los proyectos pueden verse afectados por cambios regulatorios o económicos, esta flexibilidad es invaluable.
- Colaboración con el Cliente: La colaboración constante con el cliente es central en el enfoque ágil. Esto asegura que el producto final esté alineado con las expectativas y necesidades del cliente en todo momento. Un ejemplo podría ser trabajar directamente con una cooperativa de productores nicaragüenses para desarrollar un sistema de gestión de producción, realizando ajustes basados en sus comentarios en cada sprint.

Ejemplos de Metodologías Ágiles

Entre las metodologías ágiles más conocidas se encuentran Scrum, Kanban y Extreme Programming (XP):

- Scrum: En Scrum, el trabajo se organiza en sprints, que son ciclos de desarrollo cortos (normalmente de dos a cuatro semanas). Durante cada sprint, se seleccionan tareas específicas del backlog (lista de pendientes) que se completarán. Al final de cada sprint, el equipo presenta un incremento funcional del producto. Este enfoque es ideal para proyectos que requieren una entrega rápida y continua, como el desarrollo de sistemas de gestión para pequeñas y medianas empresas en Nicaragua.
- Kanban: Kanban es una metodología que visualiza el trabajo mediante un tablero con columnas que representan las diferentes etapas del proceso de desarrollo. Las tareas se mueven a través de las columnas a medida que avanzan en su desarrollo. Este método es muy útil en entornos donde el flujo de trabajo necesita ser constantemente ajustado y optimizado, como en proyectos de mantenimiento y actualización de software existentes.
- Extreme Programming (XP): XP se centra en la mejora continua de la calidad del software mediante prácticas como la programación en parejas, pruebas automatizadas y la integración continua. Es particularmente útil en proyectos donde la calidad y la adaptabilidad del código son cruciales, como en el desarrollo de sistemas críticos para la gestión de recursos en sectores como la salud o la educación en Nicaragua.

Aplicación de Metodologías Ágiles en Proyectos Nicaragüenses

Considerando un proyecto para desarrollar una plataforma de comercio electrónico destinada a pequeñas empresas en Nicaragua, la aplicación de una metodología ágil como Scrum permitiría al equipo de desarrollo entregar funcionalidades críticas rápidamente, como la gestión de productos y la integración con métodos de pago locales. A medida que el negocio crece o cambia, la metodología ágil permitiría realizar ajustes y añadir nuevas características sin la necesidad de replantear el proyecto desde cero.

Las Metodologías Ágiles ofrecen una forma de desarrollo de software que es rápida, flexible y centrada en el cliente, ideal para entornos dinámicos como el nicaragüense. A través de la implementación de ciclos de desarrollo cortos y la constante adaptación a los cambios, los estudiantes aprenderán cómo estas metodologías pueden ser aplicadas para responder efectivamente a los desafíos locales. Al final de esta sección, los estudiantes estarán equipados con el conocimiento y las habilidades para aplicar enfoques ágiles en sus propios proyectos, asegurando que los productos que desarrollen no solo cumplan con los requisitos técnicos, sino que también estén alineados con las necesidades cambiantes de los clientes y el mercado.

Bibliografía y Webgrafía

- *Blé Jurado, C. (2023). Diseño ágil con TDD: Una introducción práctica a las pruebas de software automatizadas* (1.ª ed.). Savvily Editorial S.L.U.
- *Martel, A. (2014). Gestión práctica de proyectos con Scrum: Desarrollo de software ágil para el Scrum Master* (Aprender a ser mejor gestor de proyectos) (3.ª ed.). CreateSpace Independent Publishing Platform

Glosario

- **Ágil:** Método de desarrollo de software que enfatiza la flexibilidad, la colaboración continua y la entrega iterativa de funcionalidades, permitiendo adaptarse a cambios de requisitos de manera rápida.
- **Backlog:** Lista priorizada de tareas, requisitos o historias de usuario que deben completarse en un proyecto. En metodologías ágiles como Scrum, el backlog se utiliza para planificar y gestionar el trabajo a realizar en los sprints.
- **Cascada:** Modelo de desarrollo de software en el que las fases del proyecto se completan de manera secuencial y no permiten retrocesos a etapas anteriores. Se caracteriza por una planificación y ejecución rigurosas y lineales.
- **Colaboración:** Proceso de trabajar juntos hacia un objetivo común. En el contexto de metodologías ágiles, se refiere a la interacción continua entre el equipo de desarrollo y los clientes o usuarios para ajustar el producto según sus necesidades.
- **Desarrollo Incremental:** Enfoque en el que el software se desarrolla en pequeñas partes funcionales o incrementos, los cuales se añaden y mejoran de manera continua a lo largo del proyecto.
- **Entregas Frecuentes:** Práctica de entregar versiones funcionales del software en ciclos cortos y regulares. Esto permite a los clientes revisar y proporcionar retroalimentación continua sobre el producto.
- **Iteración:** Ciclo de trabajo en el que se desarrolla una versión del producto, se prueba y se mejora basada en retroalimentación y descubrimientos. En metodologías ágiles, las iteraciones se utilizan para refinar y ajustar el software.
- **Modelo Evolutivo:** Metodología de desarrollo que permite la evolución continua del software mediante la creación de versiones incrementales que

se ajustan a nuevos requisitos y cambios en el entorno durante el ciclo de vida del proyecto.

- **Prototipo:** Versión preliminar de un software que se utiliza para visualizar y evaluar las funcionalidades y requisitos del producto. Los prototipos permiten realizar ajustes y mejoras antes del desarrollo final.
- **Scrum:** Marco de trabajo ágil que organiza el desarrollo en ciclos cortos llamados sprints. Incluye roles específicos, como el Scrum Master y el Product Owner, y se centra en la entrega continua de valor mediante reuniones diarias y revisiones al final de cada sprint.
- **Sprints:** Ciclos de trabajo en Scrum que suelen durar entre dos y cuatro semanas. Durante un sprint, se completa una parte del proyecto y se entrega un incremento funcional del software.
- **Testing:** Proceso de verificación y validación del software para asegurar que cumple con los requisitos y funciona correctamente. En el desarrollo ágil, el testing continuo y automatizado es esencial para garantizar la calidad del producto.
- **Usuarios Finales:** Personas que utilizan el software una vez que ha sido completado e implementado. En metodologías ágiles, los usuarios finales participan activamente en la evaluación y retroalimentación del producto durante el desarrollo.
- **XP** (Extreme Programming): Metodología ágil que enfatiza la programación en parejas, la integración continua, las pruebas automatizadas y la mejora continua del código para mejorar la calidad del software y la satisfacción del cliente.